

6: Research Bot: Autonomous Information Gatherer

Implement an agent that fetches current research on a given AI topic (e.g., "latest trends in prompt engineering") using web search (SerpAPI/Bing). Summarize and present findings.

Contents

1. Program Statement
2. Aim
3. Learning Objectives
4. Software and API Requirements
5. Theoretical Background.
6. Agentic Workflow
7. Algorithm
8. Program Code
9. Explanation of Code
10. Sample Input and Output
11. Result
12. Viva Questions
13. References

1. Program Statement

6: Research Bot: Autonomous Information Gatherer

Implement an agent that fetches current research on a given AI topic, for example "latest trends in prompt engineering", using web search such as SerpAPI/Bing. Summarize and present the findings.

In this manual, the implementation is done using Google Colab, SerpAPI, and Gemini API.

2. Aim

To implement a simple Research Bot that accepts an AI research topic from the user, gathers current information from the web using SerpAPI, summarizes the collected information using Gemini, and presents the findings in a structured format.

3. Learning Objectives

- To understand how an agent can use an external web search tool.
- To learn how to collect current information from web search results.
- To understand how search results can be passed to an LLM for summarization.
- To generate a structured research summary with source links.
- To understand a simple agentic workflow involving input, tool use, reasoning, and output.

4. Software and API Requirements

Requirement	Purpose
Google Colab	To run the Python program online without local installation.
Python	To write the logic of the Research Bot.
requests library	To send the web search request to SerpAPI.
SerpAPI key	To fetch Google search results in structured JSON format.
Gemini API key	To summarize the collected research information using an LLM.
google-gemai library	To access Gemini models from Python.

5. Theoretical Background.

5.1 What is a Research Bot?

A Research Bot is an AI-based assistant that collects information about a given topic and prepares a summary. In this experiment, the bot searches the web for recent information related to an AI topic and then uses an LLM to summarize the collected results.

5.2 Why is this Agentic AI?

This program is agentic because the system performs multiple steps after receiving the user topic. It uses a tool, collects information, sends the information to an LLM, and produces a final structured answer. The bot is not only generating text from stored knowledge; it is gathering information from an external source before answering.

5.3 Role of SerpAPI

SERP stands for **Search Engine Results Page**. SerpAPI is used as the web search tool. It sends the topic to Google search and returns the results in a structured JSON format. From this response, the program extracts the title, link, and short description of each search result.

5.4 Role of Gemini

Gemini is used as the summarization model. The collected search results are given to Gemini through a prompt, and Gemini prepares a simple research summary with sections such as latest trends, applications, challenges, future scope, and conclusion.

6. Agentic Workflow

The workflow of the Research Bot is shown below:

1. User enters an AI research topic.
2. The bot sends the topic to SerpAPI for web search.
3. SerpAPI returns current search results.
4. The bot extracts titles, links, and descriptions from the results.
5. The bot sends the extracted information to Gemini.
6. Gemini summarizes the research findings.
7. The bot displays the final research summary and source links.

Stage	Action Performed
Input	The user enters a research topic.
Tool Use	SerpAPI is called to fetch web search results.
Information Extraction	Title, link, and snippet are extracted from search results.
Reasoning/Summarization	Gemini summarizes the collected information.

Output	The final research report is displayed with sources.
--------	--

7. Algorithm

1. Start the program.
2. Install and import the required libraries.
3. Read the SerpAPI key and Gemini API key securely.
4. Accept the AI research topic from the user.
5. Create a web search query using the topic.
6. Send the query to SerpAPI.
7. Collect the top five search results.
8. Extract title, link, and description from each result.
9. Create a summarization prompt using the extracted information.
10. Send the prompt to Gemini.
11. Display the structured research summary.
12. Display the source links.
13. Stop the program.

8. Program Code

Each logical block is written in a separate Google Colab cell. This helps understand the program step by step and makes debugging easier.

1: Install Required Libraries

```
# Install required libraries  
  
!pip install -q requests google-genai
```

Purpose: Installs the libraries needed for web requests and Gemini API access.

2: Import Required Libraries

```
# Import required libraries

import requests
from getpass import getpass
from google import genai
from IPython.display import Markdown, display
```

Purpose: Imports the modules needed for API calls, secure key input, Gemini access, and formatted display.

3: Enter API Keys Securely

```
#Enter API keys securely

SERPAPI_KEY = getpass("Enter SerpAPI API Key: ")
GEMINI_KEY = getpass("Enter Gemini API Key: ")
client = genai.Client(api_key=GEMINI_KEY)
```

Purpose: Reads the API keys securely and creates the Gemini client. official SerpAPI website: serpapi.com.

4: Take Research Topic from User

```
# Take research topic from user

topic = input("Enter an AI research topic: ")
```

Purpose: Accepts the research topic from the user.

5: Search the Web Using SerpAPI

```
# Search the web using SerpAPI

url = "https://serpapi.com/search"

params = {
    "engine": "google",
    "q": f"latest research trends in {topic}",
    "api_key": SERPAPI_KEY
}

response = requests.get(url, params=params)
data = response.json()

results = data.get("organic_results", [])[:5]

print("Results collected:", len(results))
```

Purpose: *Sends the topic to SerpAPI and collects the top five organic search results.*

6: Convert Search Results into Text

```
#Convert search results into text

research_text = ""

for i, item in enumerate(results, start=1):
    title = item.get("title", "No title")
    link = item.get("link", "No link")
    snippet = item.get("snippet", "No description")

    research_text += f"""
Source {i}
```

```
Title: {title}
Link: {link}
Summary: {snippet}
"""
```

Purpose: *Converts the collected search results into a simple text format that can be sent to the LLM.*

7: Summarize Research Findings Using Gemini

```
#Summarize research findings using Gemini
```

```
if results:
```

```
    prompt = f"""
```

```
    You are a Research Bot.
```

```
    Topic:
```

```
    {topic}
```

```
    Web search results:
```

```
    {research_text}
```

```
    Prepare a simple research summary with these sections:
```

```
    1. Introduction
```

```
    2. Latest Trends
```

```
    3. Applications
```

```
    4. Challenges
```

```
    5. Future Scope
```

```
    6. Conclusion
```

```
    Use beginner-friendly language.
```

```
    """
```

```
    response = client.models.generate_content(
```

```
        model="gemini-2.5-flash",
```

```
        contents=prompt
```

```
    )
```

```
    display(Markdown(response.text))
```

```
else:
```

```
    print("No results found. Please try another topic.")
```

Purpose: *Sends the collected information to Gemini and displays the structured research summary.*

8: Display Source Links

```
# Display source links
display(Markdown("## Source Links"))

for i, item in enumerate(results, start=1):
    title = item.get("title", "Source")
    link = item.get("link", "#")
    display(Markdown(f"{i}. [{title}]({link})"))
```

Purpose: *Displays the source links separately so that the user can refer to the original web pages.*

9. Explanation of Code

Cell	Explanation
Cell 1	Installs the required packages. requests is used for web search requests. google-genai is used to access Gemini.
Cell 2	Imports all required modules. getpass hides API keys while typing. Markdown and display are used for better Colab output.
Cell 3	Reads the SerpAPI key and Gemini key from the user. Then it creates a Gemini client object.
Cell 4	Accepts the AI research topic from the user. Example: latest trends in prompt engineering.
Cell 5	Creates a search query and sends it to SerpAPI. The result is received in JSON format. The top five organic results are selected.
Cell 6	Extracts the title, link, and short description from each search result and stores them in one text variable.
Cell 7	Creates a prompt and sends the collected web information to Gemini. Gemini generates a structured research summary.
Cell 8	Displays the source links separately for verification and further reading.

10. Sample Input and Expected Output

10.1 Sample Input

Sample Input

Enter an AI research topic: **Multimodal LLMs**

10.2 Output

The output may vary because the program fetches current search results from the web. However, the answer should generally appear in the following format:

Research Summary: Multimodal LLMs

1. Introduction

Large Language Models (LLMs) have revolutionized how we interact with text, but the world is more than just words. This is where **Multimodal LLMs (MM-LLMs)** come in. Think of them as advanced LLMs that can understand and generate information across various types of data, not just text. Imagine a model that can process images, audio, video, and text all together, just like humans do. This ability to handle multiple "modes" of information is what makes MM-LLMs incredibly powerful and a hot topic in AI research today. Surveys are being conducted to help guide and facilitate further research in this exciting area.

2. Latest Trends

The field of Multimodal LLMs is rapidly evolving, with significant advancements expected in the near future, specifically around 2025 and 2026. The leading edge of MM-LLMs now includes models capable of understanding and generating content across **text, image, audio, and video**. This means they can process a visual scene, listen to spoken commands, read a document, and then respond or create new content that combines these elements. Key LLM models like GPT-5, Gemini 3, Claude 4, and Llama 4 are at the forefront of these trends, pushing the boundaries of what's possible by incorporating multimodal capabilities. Research is also actively focusing on improving core aspects like reasoning abilities, how efficiently these models operate (inference-time scaling), and their fundamental architectures.

3. Applications

The ability of MM-LLMs to integrate information from different sources unlocks a wide range of practical applications:

- **Healthcare:** One exciting use case is analyzing medical images, such as **X-rays**, to assist with diagnosis or provide insights that combine visual data with patient history (text).
- **Scientific Research:** MM-LLMs are making significant progress in complex scientific domains. This includes areas like **drug discovery**, helping researchers identify new compounds; **molecular and protein design**, aiding in creating new structures with specific properties; and **materials science**, accelerating the discovery and development of new materials.
- **General Use Cases:** Beyond specialized fields, they can enhance everyday interactions, like generating video descriptions from audio and visual input, or creating rich, interactive content based on diverse user prompts.

4. Challenges

Despite their impressive capabilities, developing and deploying Multimodal LLMs comes with several challenges:

- **Design Complexity:** Creating MM-LLMs involves intricate "design formulations" to effectively integrate and process different data types. Ensuring consistent understanding and generation across various modalities (text, image, audio, video) is a significant technical hurdle.
- **Data Handling:** Training these models requires vast amounts of diverse and high-quality multimodal data, which can be difficult to collect, clean, and align.
- **Ethical Considerations and Governance:** As MM-LLMs become more powerful and integrated into various aspects of life, ensuring responsible use, addressing biases, and establishing clear guidelines for their development and deployment ("governance") become critical issues.

5. Future Scope

The future of Multimodal LLMs looks incredibly promising and is a major focus for ongoing research. We can expect:

- **Enhanced Integration:** Further improvements in how different modalities are integrated, leading to even more seamless understanding and generation.
- **Broader Applications:** Expansion into more scientific domains and real-world scenarios, finding new ways to apply their cross-modal understanding.
- **Addressing Challenges:** Continuous efforts to simplify model design, optimize training, and establish robust governance frameworks to ensure safe and ethical development.
- **Advanced Capabilities:** Research will continue to explore new reasoning models, more efficient scaling methods, and novel architectures to make MM-LLMs even more intelligent and accessible.

6. Conclusion

Multimodal LLMs represent a significant leap forward in AI, moving beyond text to understand and interact with the world in a richer, more human-like way. By combining information from text, images, audio, and video, these models are opening doors to revolutionary applications in healthcare, scientific research, and countless other fields. While challenges related to design complexity, data, and governance remain, the rapid pace of research and development indicates a future where MM-LLMs will play an increasingly pivotal role in shaping artificial intelligence.

The output will be displayed in the following format:

1. Introduction
2. Latest Trends
3. Applications
4. Challenges

5. Future Scope
6. Conclusion

Source Links

1. Source 1
2. Source 2
3. Source 3
4. Source 4
5. Source 5

Source Links

1. [LLM Research Papers: The 2025 List \(July to December\)](#)
2. [Top LLM Trends 2025: What's the Future of LLMs](#)
3. [\[2401.13601\] MM-LLMs: Recent Advances in MultiModal ...](#)
4. [Top LLMs and AI Trends for 2026 | Clarifai Industry Guide](#)
5. [A Comprehensive Survey of Multimodal LLMs for Scientific ...](#)

11. Result

The Research Bot was successfully implemented using Google Colab, SerpAPI, and Gemini API. The bot accepts an AI research topic, collects current information from web search results, summarizes the findings, and displays the final research report along with source links.

12. Viva Questions

Question	Answer
What is a Research Bot?	A Research Bot is an AI assistant that gathers information on a topic and prepares a summary.

Why is this program considered agentic?	It uses an external search tool, gathers information, processes it using an LLM, and produces a final answer.
What is the role of SerpAPI in this program?	SerpAPI is used to fetch current web search results in JSON format.
What is the role of Gemini in this program?	Gemini summarizes the collected search results into a structured research report.
What is JSON?	JSON is a structured data format used to store and exchange information between programs.
Why do we use getpass?	getpass hides the API key while typing, making the program safer to use.
What are organic_results?	organic_results are the normal search results returned by SerpAPI, usually containing title, link, and snippet.
Why do we display source links?	Source links help verify the information and allow further reading.
Can this bot replace a human researcher?	No. It can assist in gathering and summarizing information, but human verification is still required.
How can this program be improved?	It can be improved by reading full web pages, ranking sources, removing duplicates, and generating citations.

13. References

- SerpAPI Google Search API documentation: <https://serpapi.com/search-api>
- SerpAPI Organic Results documentation: <https://serpapi.com/organic-results>
- Google Gen AI Python SDK documentation: <https://googleapis-python-genai-70.mintlify.app/introduction>
- Google AI Studio / Gemini API documentation: <https://ai.google.dev/>

Note:

The focus of the program is to help clearly understand the agentic flow: user input, external tool use, information collection, LLM-based summarization, and final output presentation.